Sistema Facturación Electrónica B2B

Documentación Técnica Completa

Demian Maturana

Diciembre 2025

1. Arquitectura del Sistema

1.1. Componentes

- **Electron App (Cliente):** Aplicación desktop
- **SQLite (Local):** Base de datos local
- **Node.js Server:** Backend en Railway
- **PostgreSQL:** Base de datos servidor
- **ChileAPI:** Intermediario SII
- **SII:** Servicio de Impuestos Internos

1.2. Flujo de datos

App Cliente → SQLite (guardar local)
App Cliente ↔ Servidor (HTTPS)
Servidor → PostgreSQL
Servidor ↔ ChileAPI (API)
ChileAPI ↔ SII

1.3. Stack Tecnológico Detallado

Componente	Tecnología	Versión/Detalles
Runtime	Node.js	v20.x LTS
Desktop Framework	Electron	v28.x
UI Library	React	v18.x
Build Tool	Vite	v5.x
State Management	React Hooks	useState, useContext
DB Local	SQLite3	v5.x
DB Servidor	PostgreSQL	v15.x
ORM/Query	better-sqlite3	Síncrono, más rápido
HTTP Client	fetch API	Nativo
PDF Generation	jsPDF	v2.x
Excel Import	xlsx	SheetJS
UUID	uuid	v9.x
Packaging	electron-builder	v24.x
Backend Framework	Express	v4.x
Process Manager	PM2	Solo en producción
Task Scheduler	node-cron	v3.x
Environment	dotenv	v16.x

2. Plan de Implementación Detallado

2.1. Descripción General

Durante las próximas 6 semanas, implementarás un sistema de facturación electrónica desktop completo. El desarrollo seguirá una metodología incremental, construyendo funcionalidad básica primero y agregando complejidad progresivamente.

Filosofía de desarrollo:

- **Hacer funcionar primero, optimizar después**
- **Testing manual continuo** (cada feature que implementes, probala inmediatamente)

- **Commits frecuentes** (mínimo 2-3 por día)
- **Si algo no funciona después de 2 horas, pregunta/investiga**

2.2. Semana 1-2: Fundación

2.2.1. Objetivo

Establecer la base del sistema: estructura del proyecto, base de datos funcionando, y CRUD completo de clientes y productos con interfaz básica.

2.2.2. Día 1-2: Setup Proyecto

Qué harás:

1. Crear estructura de carpetas del proyecto
2. Inicializar npm en `electron-app/`
3. Instalar dependencias base (Electron, React, Vite)
4. Configurar `package.json` con scripts
5. Crear `vite.config.js`
6. Crear archivo `main.js` básico que abra ventana
7. Probar que Electron abre ventana vacía

Resultado esperado: Ventana Electron que abre y muestra "Hello World"
Código a implementar:

```
// src/main/main.js (versi n inicial)
import { app, BrowserWindow } from 'electron';

let mainWindow;

function createWindow() {
  mainWindow = new BrowserWindow({
    width: 1200,
    height: 800
  });

  mainWindow.loadURL('http://localhost:5173'); // Vite dev server
}

app.whenReady().then(() => {
  createWindow();
});

app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') {
    app.quit();
  }
});
```

2.2.3. Día 3-4: Base de Datos SQLite

Qué harás:

1. Instalar `better-sqlite3`
2. Crear `database/connection.js`

3. Implementar función `initDatabase()` que crea tablas
4. Crear tabla `clientes` con campos básicos
5. Probar que DB se crea correctamente
6. Implementar función `getDatabase()` para acceder a DB

Resultado esperado: Archivo `facturacion.db` creado en `AppData` con tabla clientes
Validación: Abrir DB con DB Browser for SQLite y ver que tabla existe

2.2.4. Día 5-7: CRUD Clientes

Qué harás:

1. Crear `ipc/clientes.js` con handlers:
   - `clientes:create`
   - `clientes:getAll`
   - `clientes:update`
   - `clientes:delete`
   - `clientes:search`
2. Configurar `preload.js` para exponer API
3. Crear página React `Clientes.jsx` con:
   - Formulario crear cliente (RUT, razón social, email, teléfono)
   - Tabla lista clientes
   - Botones editar/eliminar
   - Barra búsqueda
4. Crear hook `useClientes.js`
5. Probar flujo completo: crear, listar, buscar, editar, eliminar

Resultado esperado: Poder gestionar clientes completamente desde la UI
Testing manual:

- Crear 5 clientes de prueba
- Buscar por nombre
- Editar uno
- Eliminar uno
- Verificar cambios en DB con DB Browser

2.2.5. Día 8-10: CRUD Productos

Qué harás:

1. Crear tabla `productos` en DB
2. Implementar handlers IPC (similar a clientes)
3. Crear página React `Productos.jsx`
4. Agregar campos: código, nombre, precio, stock
5. Implementar validaciones (precio >0, stock $>= 0$)
6. Agregar filtro por categoría

Resultado esperado: CRUD productos funcionando igual que clientes

2.2.6. Día 11-14: Importación Excel

Qué harás:

1. Instalar librería `xlsx`
2. Crear botón Ïmportar desde Excel.[en] Clientes y Productos
3. Implementar lógica:
   - Usuario selecciona archivo .xlsx
   - Sistema lee filas
   - Valida datos (RUT válido, precios positivos, etc.)
   - Inserta en DB con `INSERT OR IGNORE`
   - Muestra resumen: X importados, Y duplicados
4. Crear Excel de prueba con 20 clientes
5. Probar importación

Resultado esperado: Poder importar masivamente clientes/productos desde Excel
Formato Excel esperado:

```
Clientes.xlsx:
RUT         | Razón Social      | Email           | Teléfono
12345678-9  | Ferretería López  | lopez@email.com | 912345678
23456789-0  | Distribuidora XYZ | xyz@email.com   | 987654321
```

2.3. Semana 3-4: Facturación Core

2.3.1. Objetivo

Implementar la funcionalidad central: crear facturas, generar XML/PDF, y preparar para envío al SII.

2.3.2. Día 15-17: Formulario Crear Factura

Qué harás:

1. Crear tabla `facturas` y `factura_items` en DB
2. Crear página `CrearFactura.jsx` con:
   - Selector cliente (autocomplete con búsqueda)
   - Tabla productos (agregar múltiples items)
   - Input cantidad por producto
   - Cálculo automático subtotales
   - Cálculo automático IVA (19 %)
   - Total general
   - Botón "Guardar Factura"
3. Implementar lógica descuento stock automático
4. Validar stock suficiente antes de crear factura

Resultado esperado: Formulario que permite crear factura seleccionando cliente y agregando productos

Testing:

- Crear factura con 3 productos
- Verificar cálculo IVA correcto
- Verificar stock descontado en tabla productos
- Verificar datos guardados en tabla facturas

2.3.3. Día 18-20: Generación Folio y Sistema Idempotencia

Qué harás:

1. Implementar generador de folios (auto-increment desde 1)
2. Guardar último folio usado en tabla `config`
3. Implementar generación `idempotency_key` (UUID v4)
4. Agregar validación: no permitir duplicar idempotency key
5. Crear función `getNextFolio()`
6. Al crear factura:
   - Generar folio
   - Generar idempotency key
   - Guardar con estado `'PENDIENTE'`

Resultado esperado: Cada factura tiene folio único correlativo
Testing:

- Crear 5 facturas seguidas
- Verificar folios: 1, 2, 3, 4, 5
- Reiniciar app, crear otra factura
- Verificar folio continúa en 6

2.3.4. Día 21-24: Generación XML DTE

Qué harás:

1. Estudiar formato XML DTE del SII (documentación ChileAPI)
2. Crear función `generarXML(factura)`
3. Implementar template XML con:
   - Datos emisor (tu RUT, razón social)
   - Datos receptor (cliente)
   - Items con cantidad, precio, subtotal
   - Totales (neto, IVA, total)
   - Fecha emisión
   - Folio
4. Guardar XML en carpeta `userData/facturas/`
5. Guardar path en columna `xml_path`

Resultado esperado: Por cada factura creada, se genera archivo XML válido
Testing:

- Crear factura
- Abrir XML generado en editor texto
- Verificar todos los datos correctos
- (Opcional) Validar XML con herramienta online

2.3.5. Día 25-28: Generación PDF

Qué harás:

1. Instalar `jspdf`
2. Crear función `generarPDF(factura)`
3. Diseñar layout PDF:
   - Header: Logo, datos emisor, folio
   - Datos cliente
   - Tabla items (código, descripción, cantidad, precio, total)
   - Totales (neto, IVA, total)
   - Footer: fecha, condiciones pago

4. Guardar PDF en `userData/facturas/`
5. Botón "Ver PDF"que abre PDF con visor sistema

Resultado esperado: PDF profesional que cliente puede imprimir
Testing:

- Generar PDF de factura de prueba
- Abrir PDF, verificar se ve bien
- Imprimir en impresora/guardar como PDF

2.4. Semana 5: Features Complementarias

2.4.1. Objetivo

Agregar funcionalidades que complementan facturación: guías de despacho, ventas a crédito, y reportes básicos.

2.4.2. Día 29-31: Guías de Despacho

Qué harás:

1. Crear tabla `guias_despacho`
2. Crear formulario similar a factura pero sin precios
3. Generar PDF guía (sin montos, solo cantidades)
4. Sistema folio independiente para guías
5. Opción: vincular guía a factura existente

Resultado esperado: Poder generar guías de despacho independientes o vinculadas

2.4.3. Día 32-34: Ventas a Crédito

Qué harás:

1. Agregar campo `tipo_pago` a facturas (EFECTIVO/CREDITO)
2. Si crédito: agregar campo `fecha_vencimiento`
3. Crear tabla `pagos_credito`
4. Página Çuentas por Cobrar":
   - Lista facturas crédito pendientes
   - Ordenadas por fecha vencimiento
   - Marcar vencidas en rojo
   - Botón Registrar Pago"(parcial o total)
5. Al pagar: actualizar `estado_pago`

Resultado esperado: Dashboard que muestra deudas pendientes y permite registrar pagos
Testing:

- Crear 3 facturas a crédito (vencimientos diferentes)
- Una con vencimiento pasado (aparece roja)
- Registrar pago parcial de una
- Verificar saldo restante correcto

2.4.4. Día 35-38: Dashboard y Reportes

Qué harás:

1. Página Dashboard con:
   - Total vendido hoy
   - Total vendido mes actual
   - Facturas pendientes envío SII (count)
   - Créditos por vencer esta semana
   - Productos bajo stock mínimo
2. Reporte ventas diarias (tabla por día)
3. Reporte ventas mensuales (gráfico simple)
4. Reporte productos más vendidos

Resultado esperado: Dashboard informativo al abrir app

2.5. Semana 6: Servidor, Licencias y Deploy

2.5.1. Objetivo

Implementar servidor backend para validación licencias y envío batch al SII. Generar instalador y hacer beta test real.

2.5.2. Día 39-41: Servidor Backend

Qué harás:

1. Crear proyecto `server/`
2. Inicializar Express
3. Crear schema PostgreSQL (licenses, processed_invoices)
4. Implementar endpoints:
   - POST /licenses/validate (valida license key + machine ID)
   - POST /invoices/batch (recibe facturas pendientes)
5. Deploy a Railway:
   - Crear cuenta Railway
   - Conectar repo GitHub
   - Configurar PostgreSQL addon
   - Deploy automático

Resultado esperado: Servidor corriendo en Railway con DB PostgreSQL
Testing:

- Llamar endpoint /health desde Postman
- Insertar licencia de prueba en DB
- Validar desde Postman

2.5.3. Día 42-43: Sistema Licencias

Qué harás:

1. En app Electron:
   - Al abrir, chequear licencia local
   - Si no existe, mostrar ventana activación
   - Usuario ingresa license key
   - App llama servidor para validar
   - Servidor registra machine ID
   - Guarda license key local si válida
2. Generar script para crear licencias:
   - `node generate-license.js Çliente XYZ"`
   - Genera UUID como license key
   - Inserta en DB con expires 30 días

Resultado esperado: App solo funciona con licencia válida

2.5.4. Día 44-45: Cron Job Envío SII

Qué harás:

1. Instalar `node-cron`
2. Crear job que corre cada hora:
   - Query facturas estado PENDING
   - Por cada una: llamar ChileAPI
   - Si éxito: actualizar estado SENT
   - Si error: actualizar estado ERROR + mensaje
3. En app cliente:
   - Botón "Sincronizar estados SII"
   - Llama servidor para obtener estados actualizados
   - Actualiza DB local

Resultado esperado: Facturas se envían automáticamente al SII cada hora

2.5.5. Día 46-47: Electron Builder

Qué harás:

1. Configurar `electron-builder.yml`
2. Agregar icono aplicación
3. Generar instalador Windows:
   - `npm run build`
   - Genera Setup.exe en carpeta dist/
4. Probar instalación en PC limpia (VM o PC familiar)
5. Verificar que crea accesos directos
6. Verificar que DB se crea correctamente

Resultado esperado: Instalador profesional funcionando

2.5.6. Día 48-50: Beta Test Real

Qué harás:

1. Instalar en PC de tu tío
2. Capacitación 1-2 horas:
   - Importar sus clientes existentes
   - Importar sus productos
   - Crear 5 facturas de prueba
   - Mostrar cómo ver PDF
   - Explicar créditos
3. Dejarlo usar 1 semana completa
4. Pedirle feedback detallado:

- ¿Qué es confuso?
- ¿Qué falta?
- ¿Qué es lento?
- ¿Bugs encontrados?

5. Iterar basado en feedback

Resultado esperado:

- Tío usa sistema real en negocio
- Lista de bugs/mejoras documentada
- Fixes aplicados
- Cliente beta satisfecho = listo para vender a otros

3. Estructura Detallada del Proyecto

3.1. Estructura Completa de Carpetas

```
facturacion-app/

          electron-app/                    # Aplicaci n cliente (Electron)
                src/
                      main/                    # Proceso principal Electron
                            main.js             # Entry point
                            database/
                                  connection.js  # Setup SQLite
                                  migrations.js  # Schema inicial
                                  queries.js     # Queries reutilizables
                            ipc/
                                  clientes.js    # Handlers clientes
                                  productos.js   # Handlers productos
                                  facturas.js    # Handlers facturas
                                  licencias.js   # Handlers licencias
                            services/
                                  pdf-generator.js
                                  xml-generator.js
                                  excel-importer.js
                            utils/
                                validators.js
                                formatters.js

                      renderer/              # UI React
                            src/
                                  App.jsx
                                  main.jsx       # Entry point React
                                  pages/
                                        Dashboard.jsx
                                        Clientes/
                                              index.jsx
                                              ClienteForm.jsx
                                              ClienteList.jsx
                                              ClienteImport.jsx
                                        Productos/
                                              index.jsx
                                              ProductoForm.jsx
                                              ProductoList.jsx
                                              ProductoImport.jsx
                                        Facturas/
                                              index.jsx
                                              FacturaForm.jsx
                                              FacturaList.jsx
                                              FacturaDetalle.jsx
                                        Creditos/
                                              index.jsx
                                              CreditosDashboard.jsx
                                              PagoCredito.jsx
                                        Reportes/
                                              index.jsx
                                              VentasDiarias.jsx
                                              VentasMensuales.jsx
                                        Configuracion/
                                            index.jsx
                                            Licencia.jsx
                                  components/
                                        common/
                                              Button.jsx
                                              Input.jsx
                                              Select.jsx
                                              Table.jsx
                                              Modal.jsx
```

```
                                        Loading.jsx
                                  layout/
                                        Navbar.jsx
                                        Sidebar.jsx
                                        Layout.jsx
                                  forms/
                                      ClienteSelector.jsx
                                      ProductoSelector.jsx
                                      RutInput.jsx
                            hooks/
                                  useClientes.js
                                  useProductos.js
                                  useFacturas.js
                                  useLicencia.js
                            context/
                                  AppContext.jsx
                            utils/
                                  formatters.js
                                  validators.js
                                  constants.js
                            styles/
                                index.css
                                tailwind.css
                      index.html
                      vite.config.js
                      package.json

            preload/
                preload.js            # Bridge seguro main <-> renderer

      assets/
            icons/
                  icon.ico             # Windows
                  icon.icns            # Mac
                  icon.png             # Linux
            images/

      public/                       # Assets est ticos

      package.json
      electron-builder.yml          # Config packaging
      README.md

server/                           # Backend Node.js
      src/
            index.js                # Entry point
            config/
                  database.js          # PostgreSQL connection
                  env.js               # Environment vars
            routes/
                  index.js             # Router principal
                  licenses.js          # GET/POST /licenses
                  invoices.js          # POST /invoices
                  telemetry.js         # POST /telemetry
            controllers/
                  licenseController.js
                  invoiceController.js
                  telemetryController.js
            services/
                  chileapi.js          # Wrapper ChileAPI
                  email.js             # Env o emails
                  machineId.js         # Validaci n fingerprinting
            db/
                  schema.sql           # Schema PostgreSQL
                  migrations/          # Migraciones futuras
                  queries.js           # Queries reutilizables
```

```
                middleware/
                      auth.js              # Validaci n license key
                      errorHandler.js
                      logger.js
                cron/
                      sendInvoices.js      # Batch env o SII (cada hora)
                      cleanupLogs.js       # Limpieza logs viejos
                utils/
                    validators.js
                    helpers.js

          tests/                           # Tests unitarios (opcional MVP)
                unit/
                integration/

          logs/                            # Logs (git ignored)

          package.json
          .env.example
          .gitignore
          README.md

    docs/
          ARCHITECTURE.pdf                 # Este documento
          API.md                           # Documentaci n API servidor
          DATABASE.md                      # Schema detallado
          DEPLOYMENT.md                    # Gu a deploy

    scripts/                              # Scripts  tiles
          db-setup.sql                     # Setup DB desarrollo
          generate-license.js              # Generar license keys

    .gitignore
    README.md
```

4. Configuración Inicial Paso a Paso

4.1. Electron App

1. Inicializar proyecto:

```
mkdir electron-app
cd electron-app
npm init -y
```

2. Instalar dependencias principales:

```
# Core
npm install electron

# UI
npm install react react-dom
npm install -D vite @vitejs/plugin-react

# Database
npm install better-sqlite3

# Utilities
npm install uuid
npm install jspdf
npm install xlsx

# Build
npm install -D electron-builder
```

3. package.json completo:

```
{
  "name": "sistema-facturacion",
  "version": "1.0.0",
  "description": "Sistema de facturacion electronica",
  "main": "src/main/main.js",
  "scripts": {
    "dev": "vite",
    "build:renderer": "vite build",
    "build": "npm run build:renderer && electron-builder",
    "start": "electron ."
  },
  "dependencies": {
    "better-sqlite3": "^9.2.2",
    "jspdf": "^2.5.1",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "uuid": "^9.0.1",
    "xlsx": "^0.18.5"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.2.1",
    "electron": "^28.1.0",
    "electron-builder": "^24.9.1",
    "vite": "^5.0.8"
  }
}
```

4. vite.config.js:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import path from 'path';

export default defineConfig({
  plugins: [react()],
  base: './',
  build: {
```

```
    outDir: 'dist-renderer',
    emptyOutDir: true
  },
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src/renderer/src')
    }
  }
});
```

5. **electron-builder.yml:**

```
appId: cl.facturacion.app
productName: Sistema Facturacion
copyright: Copyright    2025

directories:
  output: dist
  buildResources: assets

files:
  - dist-renderer/**/*
  - src/main/**/*
  - src/preload/**/*
  - package.json

win:
  target:
    - target: nsis
      arch:
        - x64
  icon: assets/icons/icon.ico

nsis:
  oneClick: false
  allowToChangeInstallationDirectory: true
  createDesktopShortcut: true
  createStartMenuShortcut: true
  shortcutName: Sistema Facturacion

mac:
  target:
    - dmg
  icon: assets/icons/icon.icns
  category: public.app-category.business

linux:
  target:
    - AppImage
  icon: assets/icons/icon.png
  category: Office
```

4.2. Servidor Node.js

1. Inicializar proyecto:

```
mkdir server
cd server
npm init -y
```

2. Instalar dependencias:

```
# Core
npm install express
npm install pg
npm install dotenv

# Utilities
npm install node-cron
npm install cors
npm install helmet

# Development
npm install -D nodemon
```

3. package.json servidor:

```
{
  "name": "facturacion-server",
  "version": "1.0.0",
  "description": "Backend para sistema facturacion",
  "main": "src/index.js",
  "type": "module",
  "scripts": {
    "dev": "nodemon src/index.js",
    "start": "node src/index.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.3.1",
    "express": "^4.18.2",
    "helmet": "^7.1.0",
    "node-cron": "^3.0.3",
    "pg": "^8.11.3"
  },
  "devDependencies": {
    "nodemon": "^3.0.2"
  }
}
```

4. .env.example:

```
# Server
PORT=3000
NODE_ENV=development

# Database
DATABASE_URL=postgresql://user:password@localhost:5432/facturacion

# ChileAPI
CHILEAPI_URL=https://api.chileapi.dev
CHILEAPI_KEY=your_api_key_here

# Security
JWT_SECRET=your_secret_here
ADMIN_API_KEY=your_admin_key_here

# Email (opcional)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your_email@gmail.com
```

```
SMTP_PASS=your_password

# Cron
INVOICE_BATCH_CRON=0 * * * *  # Cada hora
```

5. Código Base Crítico

5.1. Main Process (Electron)

src/main/main.js:

```
import { app, BrowserWindow, ipcMain } from 'electron';
import path from 'path';
import { initDatabase } from './database/connection.js';
import { registerClientesHandlers } from './ipc/clientes.js';
import { registerProductosHandlers } from './ipc/productos.js';
import { registerFacturasHandlers } from './ipc/facturas.js';
import { registerLicenciasHandlers } from './ipc/licencias.js';

let mainWindow;

async function createWindow() {
  mainWindow = new BrowserWindow({
    width: 1200,
    height: 800,
    webPreferences: {
      preload: path.join(__dirname, '../preload/preload.js'),
      nodeIntegration: false,
      contextIsolation: true
    }
  });

  // Desarrollo: Vite dev server
  if (process.env.NODE_ENV === 'development') {
    mainWindow.loadURL('http://localhost:5173');
    mainWindow.webContents.openDevTools();
  } else {
    // Produccion: archivos compilados
    mainWindow.loadFile(
      path.join(__dirname, '../../dist-renderer/index.html')
    );
  }
}

app.whenReady().then(async () => {
  // Inicializar DB
  await initDatabase();

  // Registrar IPC handlers
  registerClientesHandlers();
  registerProductosHandlers();
  registerFacturasHandlers();
  registerLicenciasHandlers();

  // Validar licencia antes de abrir
  const isValid = await validateLicense();

  if (isValid) {
    createWindow();
  } else {
    createActivationWindow();
  }
});

app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') {
    app.quit();
  }
});

async function validateLicense() {
```

```
  // TODO: Implementar validacion
  return true; // Por ahora siempre true
}

function createActivationWindow() {
  // TODO: Ventana para activar licencia
}
```

5.2. Database Connection (SQLite)

src/main/database/connection.js:

```
import Database from 'better-sqlite3';
import path from 'path';
import { app } from 'electron';

let db;

export function initDatabase() {
  const userDataPath = app.getPath('userData');
  const dbPath = path.join(userDataPath, 'facturacion.db');

  db = new Database(dbPath);

  // Configuracion
  db.pragma('journal_mode = WAL');
  db.pragma('foreign_keys = ON');

  // Crear tablas si no existen
  createTables();

  console.log('Database initialized:', dbPath);

  return db;
}

export function getDatabase() {
  if (!db) {
    throw new Error('Database not initialized');
  }
  return db;
}

function createTables() {
  db.exec(`
    CREATE TABLE IF NOT EXISTS clientes (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      rut TEXT UNIQUE NOT NULL,
      razon_social TEXT NOT NULL,
      giro TEXT,
      direccion TEXT,
      comuna TEXT,
      email TEXT,
      telefono TEXT,
      credito_maximo INTEGER DEFAULT 0,
      created_at DATETIME DEFAULT CURRENT_TIMESTAMP
    );

    CREATE TABLE IF NOT EXISTS productos (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      codigo TEXT UNIQUE NOT NULL,
      nombre TEXT NOT NULL,
      descripcion TEXT,
      precio INTEGER NOT NULL,
      stock INTEGER DEFAULT 0,
      stock_minimo INTEGER DEFAULT 0,
      categoria TEXT,
      created_at DATETIME DEFAULT CURRENT_TIMESTAMP
    );

    CREATE TABLE IF NOT EXISTS facturas (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      folio INTEGER UNIQUE NOT NULL,
      idempotency_key TEXT UNIQUE NOT NULL,
      cliente_id INTEGER NOT NULL,
```

```
      fecha DATE NOT NULL,
      subtotal INTEGER NOT NULL,
      iva INTEGER NOT NULL,
      total INTEGER NOT NULL,
      tipo_pago TEXT CHECK(tipo_pago IN ('EFECTIVO', 'TARJETA', 'CREDITO')),
      fecha_vencimiento DATE,
      estado_pago TEXT DEFAULT 'PENDIENTE',
      estado_sii TEXT DEFAULT 'PENDIENTE',
      fecha_envio_sii DATETIME,
      track_id TEXT,
      error_sii TEXT,
      xml_path TEXT,
      pdf_path TEXT,
      created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
      FOREIGN KEY (cliente_id) REFERENCES clientes(id)
    );

    CREATE TABLE IF NOT EXISTS factura_items (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      factura_id INTEGER NOT NULL,
      producto_id INTEGER NOT NULL,
      cantidad INTEGER NOT NULL,
      precio_unitario INTEGER NOT NULL,
      subtotal INTEGER NOT NULL,
      FOREIGN KEY (factura_id) REFERENCES facturas(id) ON DELETE CASCADE,
      FOREIGN KEY (producto_id) REFERENCES productos(id)
    );

    CREATE TABLE IF NOT EXISTS guias_despacho (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      folio INTEGER UNIQUE NOT NULL,
      factura_id INTEGER,
      cliente_id INTEGER NOT NULL,
      fecha DATE NOT NULL,
      direccion_despacho TEXT NOT NULL,
      created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
      FOREIGN KEY (factura_id) REFERENCES facturas(id),
      FOREIGN KEY (cliente_id) REFERENCES clientes(id)
    );

    CREATE TABLE IF NOT EXISTS pagos_credito (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      factura_id INTEGER NOT NULL,
      monto INTEGER NOT NULL,
      fecha_pago DATE NOT NULL,
      metodo_pago TEXT,
      notas TEXT,
      created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
      FOREIGN KEY (factura_id) REFERENCES facturas(id)
    );

    CREATE TABLE IF NOT EXISTS config (
      key TEXT PRIMARY KEY,
      value TEXT NOT NULL
    );

    -- Indices para performance
    CREATE INDEX IF NOT EXISTS idx_clientes_rut ON clientes(rut);
    CREATE INDEX IF NOT EXISTS idx_productos_codigo ON productos(codigo);
    CREATE INDEX IF NOT EXISTS idx_facturas_folio ON facturas(folio);
    CREATE INDEX IF NOT EXISTS idx_facturas_estado_sii ON facturas(estado_sii);
    CREATE INDEX IF NOT EXISTS idx_facturas_cliente ON facturas(cliente_id);
  `);
}
```

5.3. IPC Handlers - Clientes

src/main/ipc/clientes.js:

```
import { ipcMain } from 'electron';
import { getDatabase } from '../database/connection.js';

export function registerClientesHandlers() {

  // Crear cliente
  ipcMain.handle('clientes:create', async (event, cliente) => {
    const db = getDatabase();

    try {
      const stmt = db.prepare(`
        INSERT INTO clientes (
          rut, razon_social, giro, direccion, comuna,
          email, telefono, credito_maximo
        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
      `);

      const result = stmt.run(
        cliente.rut,
        cliente.razonSocial,
        cliente.giro || null,
        cliente.direccion || null,
        cliente.comuna || null,
        cliente.email || null,
        cliente.telefono || null,
        cliente.creditoMaximo || 0
      );

      return { success: true, id: result.lastInsertRowid };
    } catch (error) {
      if (error.code === 'SQLITE_CONSTRAINT_UNIQUE') {
        throw new Error('RUT ya existe');
      }
      throw error;
    }
  });

  // Obtener todos los clientes
  ipcMain.handle('clientes:getAll', async () => {
    const db = getDatabase();
    const stmt = db.prepare('SELECT * FROM clientes ORDER BY razon_social');
    return stmt.all();
  });

  // Obtener cliente por ID
  ipcMain.handle('clientes:getById', async (event, id) => {
    const db = getDatabase();
    const stmt = db.prepare('SELECT * FROM clientes WHERE id = ?');
    return stmt.get(id);
  });

  // Buscar clientes
  ipcMain.handle('clientes:search', async (event, term) => {
    const db = getDatabase();
    const stmt = db.prepare(`
      SELECT * FROM clientes
      WHERE razon_social LIKE ? OR rut LIKE ?
      ORDER BY razon_social
      LIMIT 20
    `);
    const searchTerm = `%${term}%`;
    return stmt.all(searchTerm, searchTerm);
  });
```

```
64
65  // Actualizar cliente
66  ipcMain.handle('clientes:update', async (event, id, cliente) => {
67    const db = getDatabase();
68    const stmt = db.prepare(`
69      UPDATE clientes SET
70        razon_social = ?,
71        giro = ?,
72        direccion = ?,
73        comuna = ?,
74        email = ?,
75        telefono = ?,
76        credito_maximo = ?
77      WHERE id = ?
78    `);
79
80    stmt.run(
81      cliente.razonSocial,
82      cliente.giro,
83      cliente.direccion,
84      cliente.comuna,
85      cliente.email,
86      cliente.telefono,
87      cliente.creditoMaximo,
88      id
89    );
90
91    return { success: true };
92  });
93
94  // Eliminar cliente
95  ipcMain.handle('clientes:delete', async (event, id) => {
96    const db = getDatabase();
97
98    // Verificar si tiene facturas
99    const checkStmt = db.prepare(
100     'SELECT COUNT(*) as count FROM facturas WHERE cliente_id = ?'
101   );
102   const { count } = checkStmt.get(id);
103
104   if (count > 0) {
105     throw new Error('No se puede eliminar cliente con facturas asociadas');
106   }
107
108   const stmt = db.prepare('DELETE FROM clientes WHERE id = ?');
109   stmt.run(id);
110
111   return { success: true };
112 });
113
114 // Importar desde Excel
115 ipcMain.handle('clientes:importExcel', async (event, clientes) => {
116   const db = getDatabase();
117   const insertStmt = db.prepare(`
118     INSERT OR IGNORE INTO clientes (
119       rut, razon_social, giro, direccion, email, telefono
120     ) VALUES (?, ?, ?, ?, ?, ?)
121   `);
122
123   const transaction = db.transaction((clientesList) => {
124     for (const cliente of clientesList) {
125       insertStmt.run(
126         cliente.rut,
127         cliente.razonSocial,
128         cliente.giro || null,
129         cliente.direccion || null,
```

```
          cliente.email || null,
          cliente.telefono || null
        );
      }
    });

    try {
      transaction(clientes);
      return { success: true, imported: clientes.length };
    } catch (error) {
      throw error;
    }
  });
}
```

5.4. Preload Script

src/preload/preload.js:

```
import { contextBridge, ipcRenderer } from 'electron';

// Exponer API segura al renderer
contextBridge.exposeInMainWorld('api', {

  // Clientes
  clientes: {
    create: (cliente) => ipcRenderer.invoke('clientes:create', cliente),
    getAll: () => ipcRenderer.invoke('clientes:getAll'),
    getById: (id) => ipcRenderer.invoke('clientes:getById', id),
    search: (term) => ipcRenderer.invoke('clientes:search', term),
    update: (id, cliente) => ipcRenderer.invoke('clientes:update', id, cliente),
    delete: (id) => ipcRenderer.invoke('clientes:delete', id),
    importExcel: (clientes) => ipcRenderer.invoke('clientes:importExcel', clientes)
  },

  // Productos
  productos: {
    create: (producto) => ipcRenderer.invoke('productos:create', producto),
    getAll: () => ipcRenderer.invoke('productos:getAll'),
    getById: (id) => ipcRenderer.invoke('productos:getById', id),
    search: (term) => ipcRenderer.invoke('productos:search', term),
    update: (id, producto) => ipcRenderer.invoke('productos:update', id, producto),
    delete: (id) => ipcRenderer.invoke('productos:delete', id),
    updateStock: (id, cantidad) => ipcRenderer.invoke('productos:updateStock', id,
    cantidad),
    importExcel: (productos) => ipcRenderer.invoke('productos:importExcel', productos
    )
  },

  // Facturas
  facturas: {
    create: (factura) => ipcRenderer.invoke('facturas:create', factura),
    getAll: () => ipcRenderer.invoke('facturas:getAll'),
    getById: (id) => ipcRenderer.invoke('facturas:getById', id),
    getPendientes: () => ipcRenderer.invoke('facturas:getPendientes'),
    generarPDF: (id) => ipcRenderer.invoke('facturas:generarPDF', id),
    enviarEmail: (id, email) => ipcRenderer.invoke('facturas:enviarEmail', id, email)
  },

  // Creditos
  creditos: {
    getByCliente: (clienteId) => ipcRenderer.invoke('creditos:getByCliente',
    clienteId),
    getVencidas: () => ipcRenderer.invoke('creditos:getVencidas'),
    registrarPago: (pago) => ipcRenderer.invoke('creditos:registrarPago', pago)
  },

  // Licencias
  licencia: {
    validate: () => ipcRenderer.invoke('licencia:validate'),
    activate: (key) => ipcRenderer.invoke('licencia:activate', key),
    getStatus: () => ipcRenderer.invoke('licencia:getStatus')
  },

  // Utils
  utils: {
    openPDF: (path) => ipcRenderer.invoke('utils:openPDF', path),
    selectFile: (options) => ipcRenderer.invoke('utils:selectFile', options)
  }
});
```

5.5. React App Entry

src/renderer/src/main.jsx:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import './styles/index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

src/renderer/src/App.jsx:

```
import React, { useState } from 'react';
import { AppContext } from './context/AppContext';
import Layout from './components/layout/Layout';
import Dashboard from './pages/Dashboard';
import Clientes from './pages/Clientes';
import Productos from './pages/Productos';
import Facturas from './pages/Facturas';
import Creditos from './pages/Creditos';
import Reportes from './pages/Reportes';
import Configuracion from './pages/Configuracion';

function App() {
  const [currentPage, setCurrentPage] = useState('dashboard');
  const [user, setUser] = useState(null);

  const renderPage = () => {
    switch (currentPage) {
      case 'dashboard':
        return <Dashboard />;
      case 'clientes':
        return <Clientes />;
      case 'productos':
        return <Productos />;
      case 'facturas':
        return <Facturas />;
      case 'creditos':
        return <Creditos />;
      case 'reportes':
        return <Reportes />;
      case 'configuracion':
        return <Configuracion />;
      default:
        return <Dashboard />;
    }
  };

  return (
    <AppContext.Provider value={{ currentPage, setCurrentPage, user, setUser }}>
      <Layout>
        {renderPage()}
      </Layout>
    </AppContext.Provider>
  );
}

export default App;
```

5.6. Custom Hook - useClientes

src/renderer/src/hooks/useClientes.js:

```
import { useState, useEffect, useCallback } from 'react';

export function useClientes() {
  const [clientes, setClientes] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const loadClientes = useCallback(async () => {
    setLoading(true);
    setError(null);

    try {
      const data = await window.api.clientes.getAll();
      setClientes(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    loadClientes();
  }, [loadClientes]);

  const createCliente = async (cliente) => {
    try {
      await window.api.clientes.create(cliente);
      await loadClientes();
      return { success: true };
    } catch (err) {
      return { success: false, error: err.message };
    }
  };

  const updateCliente = async (id, cliente) => {
    try {
      await window.api.clientes.update(id, cliente);
      await loadClientes();
      return { success: true };
    } catch (err) {
      return { success: false, error: err.message };
    }
  };

  const deleteCliente = async (id) => {
    try {
      await window.api.clientes.delete(id);
      await loadClientes();
      return { success: true };
    } catch (err) {
      return { success: false, error: err.message };
    }
  };

  const searchClientes = async (term) => {
    setLoading(true);
    try {
      const data = await window.api.clientes.search(term);
      setClientes(data);
    } catch (err) {
      setError(err.message);
    } finally {
```

```
      setLoading(false);
    }
  };

  return {
    clientes,
    loading,
    error,
    createCliente,
    updateCliente,
    deleteCliente,
    searchClientes,
    refreshClientes: loadClientes
  };
}
```

6. Servidor Backend

6.1. Entry Point

server/src/index.js:

```
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import dotenv from 'dotenv';
import cron from 'node-cron';

import { initDatabase } from './config/database.js';
import routes from './routes/index.js';
import { errorHandler } from './middleware/errorHandler.js';
import { startInvoiceBatchJob } from './cron/sendInvoices.js';

dotenv.config();

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware
app.use(helmet());
app.use(cors({
  origin: process.env.ALLOWED_ORIGINS?.split(',') || '*'
}));
app.use(express.json());

// Routes
app.use('/api', routes);

// Health check
app.get('/health', (req, res) => {
  res.json({ status: 'OK', timestamp: new Date().toISOString() });
});

// Error handling
app.use(errorHandler);

// Iniciar servidor
async function start() {
  try {
    // Conectar DB
    await initDatabase();
    console.log('    Database connected');

    // Iniciar cron jobs
    startInvoiceBatchJob();
    console.log('    Cron jobs started');

    // Listen
    app.listen(PORT, () => {
      console.log(`    Server running on port ${PORT}`);
      console.log(`  Environment: ${process.env.NODE_ENV}`);
    });
  } catch (error) {
    console.error('    Failed to start server:', error);
    process.exit(1);
  }
}

start();
```

6.2. Database Config

server/src/config/database.js:

```
import pg from 'pg';
import dotenv from 'dotenv';
import fs from 'fs';
import path from 'path';
import { fileURLToPath } from 'url';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

dotenv.config();

const { Pool } = pg;

let pool;

export async function initDatabase() {
  pool = new Pool({
    connectionString: process.env.DATABASE_URL,
    ssl: process.env.NODE_ENV === 'production' ? {
      rejectUnauthorized: false
    } : false
  });

  // Test connection
  try {
    await pool.query('SELECT NOW()');

    // Run schema if tables don't exist
    await createTablesIfNotExist();

    return pool;
  } catch (error) {
    console.error('Database connection failed:', error);
    throw error;
  }
}

export function getPool() {
  if (!pool) {
    throw new Error('Database not initialized');
  }
  return pool;
}

async function createTablesIfNotExist() {
  const schemaPath = path.join(__dirname, '../db/schema.sql');
  const schema = fs.readFileSync(schemaPath, 'utf8');

  try {
    await pool.query(schema);
    console.log('    Database schema created/verified');
  } catch (error) {
    console.error('Schema creation failed:', error);
    throw error;
  }
}
```

server/src/db/schema.sql:

```
-- Licencias
CREATE TABLE IF NOT EXISTS licenses (
  id SERIAL PRIMARY KEY,
  license_key TEXT UNIQUE NOT NULL,
  client_name TEXT NOT NULL,
```

```
  client_rut TEXT NOT NULL,
  client_email TEXT,
  machine_id TEXT,
  hostname TEXT,
  active BOOLEAN DEFAULT true,
  expires_at TIMESTAMP NOT NULL,
  activated_at TIMESTAMP,
  last_seen TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Facturas procesadas
CREATE TABLE IF NOT EXISTS processed_invoices (
  id SERIAL PRIMARY KEY,
  idempotency_key TEXT UNIQUE NOT NULL,
  license_key TEXT NOT NULL,
  folio INTEGER NOT NULL,
  status TEXT NOT NULL CHECK(status IN ('PENDING', 'SENT', 'ERROR')),
  chile_api_track_id TEXT,
  chile_api_folio TEXT,
  pdf_url TEXT,
  xml_url TEXT,
  error_message TEXT,
  retry_count INTEGER DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW(),
  sent_at TIMESTAMP,
  FOREIGN KEY (license_key) REFERENCES licenses(license_key) ON DELETE CASCADE
);

-- Telemetria
CREATE TABLE IF NOT EXISTS telemetry (
  id SERIAL PRIMARY KEY,
  license_key TEXT NOT NULL,
  machine_id TEXT NOT NULL,
  action TEXT NOT NULL,
  app_version TEXT,
  metadata JSONB,
  timestamp TIMESTAMP DEFAULT NOW(),
  FOREIGN KEY (license_key) REFERENCES licenses(license_key) ON DELETE CASCADE
);

-- Alertas
CREATE TABLE IF NOT EXISTS alerts (
  id SERIAL PRIMARY KEY,
  type TEXT NOT NULL,
  license_key TEXT NOT NULL,
  severity TEXT CHECK(severity IN ('INFO', 'WARNING', 'ERROR')),
  details JSONB,
  resolved BOOLEAN DEFAULT false,
  resolved_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW(),
  FOREIGN KEY (license_key) REFERENCES licenses(license_key) ON DELETE CASCADE
);

-- Indices
CREATE INDEX IF NOT EXISTS idx_licenses_key ON licenses(license_key);
CREATE INDEX IF NOT EXISTS idx_licenses_machine ON licenses(machine_id);
CREATE INDEX IF NOT EXISTS idx_invoices_idempotency ON processed_invoices(
    idempotency_key);
CREATE INDEX IF NOT EXISTS idx_invoices_status ON processed_invoices(status);
CREATE INDEX IF NOT EXISTS idx_invoices_license ON processed_invoices(license_key);
CREATE INDEX IF NOT EXISTS idx_telemetry_license ON telemetry(license_key);
CREATE INDEX IF NOT EXISTS idx_telemetry_timestamp ON telemetry(timestamp);
CREATE INDEX IF NOT EXISTS idx_alerts_license ON alerts(license_key);
CREATE INDEX IF NOT EXISTS idx_alerts_resolved ON alerts(resolved);
```

6.3. Cron Job - Envío Batch SII

server/src/cron/sendInvoices.js:

```
import cron from 'node-cron';
import { getPool } from '../config/database.js';
import { sendToChileAPI } from '../services/chileapi.js';

export function startInvoiceBatchJob() {
  // Cada hora
  const cronExpression = process.env.INVOICE_BATCH_CRON || '0 * * * *';

  cron.schedule(cronExpression, async () => {
    console.log(`[CRON] Starting invoice batch job at ${new Date().toISOString()}`);

    try {
      await processInvoices();
    } catch (error) {
      console.error('[CRON] Batch job failed:', error);
    }
  });

  console.log(`    Invoice batch job scheduled: ${cronExpression}`);
}

async function processInvoices() {
  const pool = getPool();

  // Obtener facturas pendientes
  // Mas de 5 min de antig edad para evitar race conditions
  const result = await pool.query(`
    SELECT * FROM processed_invoices
    WHERE status = 'PENDING'
      AND created_at < NOW() - INTERVAL '5 minutes'
      AND retry_count < 5
    ORDER BY created_at ASC
    LIMIT 100
  `);

  const pendientes = result.rows;
  console.log(`[CRON] Found ${pendientes.length} pending invoices`);

  let sent = 0;
  let errors = 0;

  for (const invoice of pendientes) {
    try {
      // Llamar ChileAPI
      const apiResult = await sendToChileAPI(invoice);

      // Actualizar como enviada
      await pool.query(`
        UPDATE processed_invoices
        SET status = 'SENT',
            chile_api_folio = $1,
            chile_api_track_id = $2,
            pdf_url = $3,
            xml_url = $4,
            sent_at = NOW()
        WHERE id = $5
      `, [
        apiResult.folio,
        apiResult.trackId,
        apiResult.pdfUrl,
        apiResult.xmlUrl,
        invoice.id
      ]);
```

```
64
65        sent++;
66        console.log(`      Invoice ${invoice.folio} sent`);
67
68        // Peque a pausa entre env os
69        await sleep(500);
70
71      } catch (error) {
72        errors++;
73        console.error(`      Invoice ${invoice.folio} failed:`, error.message);
74
75        // Actualizar como error
76        await pool.query(`
77          UPDATE processed_invoices
78          SET status = 'ERROR',
79              error_message = $1,
80              retry_count = retry_count + 1
81          WHERE id = $2
82        `, [error.message, invoice.id]);
83
84        // Crear alerta si muchos reintentos
85        if (invoice.retry_count >= 3) {
86          await pool.query(`
87            INSERT INTO alerts (type, license_key, severity, details)
88            VALUES ('invoice_failed', $1, 'ERROR', $2)
89          `, [
90            invoice.license_key,
91            JSON.stringify({
92              invoiceId: invoice.id,
93              folio: invoice.folio,
94              error: error.message,
95              retries: invoice.retry_count + 1
96            })
97          ]);
98        }
99      }
100   }
101
102   console.log(`[CRON] Batch completed: ${sent} sent, ${errors} errors`);
103 }
104
105 function sleep(ms) {
106   return new Promise(resolve => setTimeout(resolve, ms));
107 }
```

7. Checklist Implementación

7.1. Semana 1-2: Base

- [ ] Setup proyecto Electron
- [ ] Configurar Vite + React
- [ ] Implementar database/connection.js
- [ ] Crear tablas SQLite
- [ ] Implementar preload.js
- [ ] Crear componentes UI base (Button, Input, Table, Modal)
- [ ] Implementar Layout (Navbar + Sidebar)
- [ ] Página Clientes - CRUD completo
- [ ] Página Productos - CRUD completo
- [ ] Importación Excel (clientes y productos)
- [ ] Testing manual funcionalidades

7.2. Semana 3-4: Facturación

- [ ] Generador XML DTE (según formato SII)
- [ ] Generador PDF facturas (jsPDF)
- [ ] Formulario crear factura (selector cliente + productos)
- [ ] Cálculo automático IVA
- [ ] Sistema folios (auto-incremento)
- [ ] Sistema idempotency keys
- [ ] Guardar facturas con estado PENDIENTE
- [ ] Lista facturas (filtros por estado)
- [ ] Detalle factura (modal)
- [ ] Actualización stock automática
- [ ] Testing flujo completo

7.3. Semana 5: Features Adicionales

- [ ] Guías de despacho (formulario + PDF)
- [ ] Ventas a crédito (campo fecha vencimiento)
- [ ] Registro pagos crédito
- [ ] Dashboard cuentas por cobrar
- [ ] Alertas deudas vencidas
- [ ] Reportes ventas diarias
- [ ] Reportes ventas mensuales
- [ ] Testing features

7.4. Semana 6: Servidor + Deploy

Setup servidor Node.js

Configurar PostgreSQL (Railway)

Implementar schema.sql

Endpoints licencias (validar, activar)

Endpoint enviar factura (POST /invoices)

Servicio ChileAPI wrapper

Cron job envío batch SII

Machine ID fingerprinting

Deploy servidor a Railway

Configurar Electron Builder

Generar instalador Windows

Testing con tío (beta real)

Fixes basados en feedback

Documentación README

8. Comandos Útiles

8.1. Desarrollo

```
# Electron App
cd electron-app
npm run dev              # Vite dev server + Electron

# Servidor
cd server
npm run dev              # Nodemon auto-reload

# Ambos simult neamente (con concurrently)
npm install -g concurrently
concurrently "cd electron-app && npm run dev" "cd server && npm run dev"
```

8.2. Build Producción

```
# Build instalador Windows
cd electron-app
npm run build:renderer   # Compilar React
npm run build            # Electron Builder

# Resultado:
# dist/Sistema-Facturacion-Setup-1.0.0.exe
```

8.3. Deploy Servidor

```
# Railway CLI
railway login
railway init
railway up

# O desde GitHub (auto-deploy)
# 1. Push a GitHub
# 2. Conectar repo en Railway dashboard
# 3. Auto-deploy en cada push
```

8.4. Database

```
# SQLite local (desarrollo)
sqlite3 ~/AppData/Roaming/sistema-facturacion/facturacion.db
.tables
SELECT * FROM clientes LIMIT 5;

# PostgreSQL remoto (Railway)
railway connect postgres
\dt
SELECT * FROM licenses;
```

9. Consideraciones Importantes

9.1. Seguridad

- **Nunca exponer API keys en código cliente**
  - ChileAPI key SOLO en servidor
  - Variables entorno (.env) para secretos
  - .gitignore para .env
- **Validar TODAS las entradas**
  - RUT format
  - Email format
  - Números positivos
  - SQL injection (usar prepared statements)
- **Ofuscar código antes de build producción**
  - javascript-obfuscator
  - Aplicar en build script

9.2. Performance

- **Indices DB**
  - Todos los campos de búsqueda indexados
  - Foreign keys indexadas
- **Paginación**
  - Listas grandes (productos, facturas) con LIMIT
  - Scroll infinito o paginación tradicional
- **Lazy loading**
  - Componentes React con React.lazy()
  - Imágenes/PDFs carga diferida

9.3. Testing

- **Manual testing crítico**
  - Flujo completo crear factura
  - Importación Excel (archivos variados)
  - Escenarios error (RUT duplicado, stock insuficiente)
- **Beta testing**
  - Tío como primer beta tester
  - Mínimo 1 semana uso real
  - Recopilar feedback detallado

9.4. Documentación

- **README.md claro**
  - Setup desarrollo
  - Comandos principales
  - Estructura proyecto
- **Comentarios código**
  - Funciones complejas
  - Lógica de negocio
  - TODOs para futuro
- **Manual usuario**
  - Screenshots flujos principales
  - FAQs
  - Troubleshooting común

10. Próximos Pasos

1. **Crear repositorio Git**

```
git init
git add .
git commit -m "Initial commit: Project structure"
git remote add origin <tu-repo>
git push -u origin main

```

2. **Setup entorno desarrollo**
   - Instalar Node.js 20.x LTS
   - Instalar VS Code + extensiones
   - Clonar estructura carpetas
3. **Empezar con lo básico**
   - Implementar database/connection.js
   - Crear tabla clientes
   - CRUD clientes funcionando
   - Después expandir a productos, facturas
4. **Iterar rápido**
   - Commits frecuentes
   - Testing manual continuo
   - Ajustar según lo que funcione

Este documento es tu guía completa.
Ahora: Ejecutar.